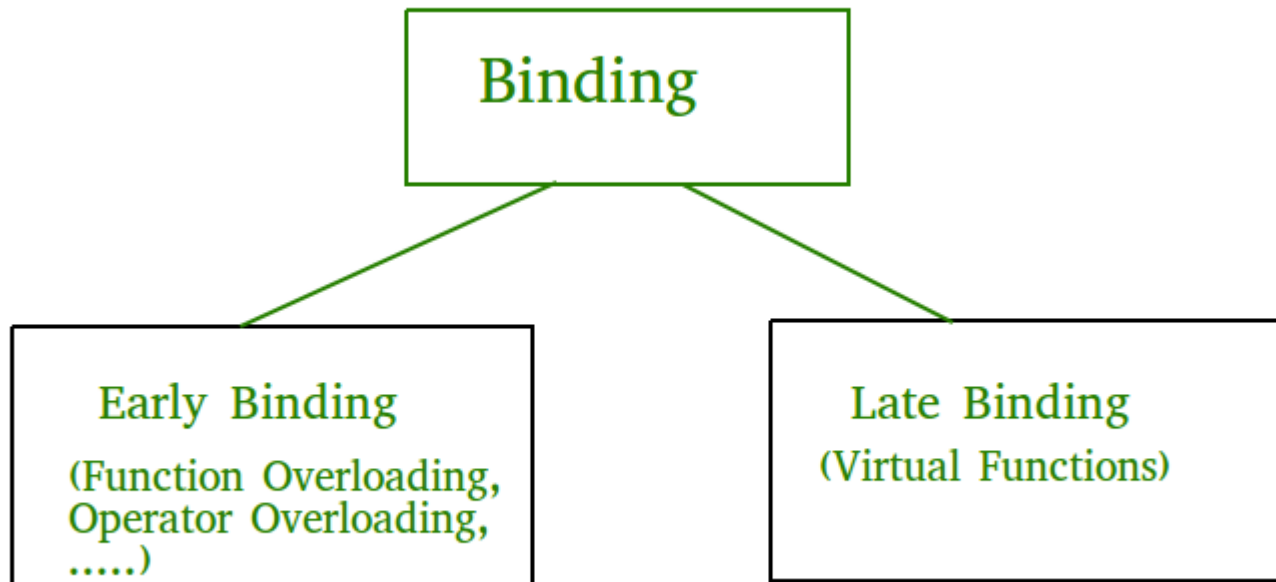


DYNAMIC BINDING

MODULE 7

STATIC BINDING

Overloaded functions are selected by the compiler by matching number and type of argument and is called early binding or static binding or static linking or compile time polymorphism.



SITUATION NEEDING DYNAMIC BINDING

```
#include<iostream>
```

```
using namespace std;
```

```
class Base
```

```
{
```

```
public:
```

```
    void show() { cout<<" In  
Base \n"; }
```

```
};
```

```
class Derived: public Base
```

```
{
```

```
public:
```

```
    void show() { cout<<"In  
Derived \n"; }
```

```
};
```

```
int main(void)
```

```
{
```

```
    Base *bp = new Derived;
```

```
    bp->show();
```

```
    return 0;
```

```
}
```

LATE BINDING

```
#include<iostream>
using namespace std;

class Base
{
public:
    virtual void show() {
        cout<<" In Base \n"; }
};
```

```
class Derived: public Base
{
public:
    void show() { cout<<"In
Derived \n"; }
};

int main(void)
{
    Base *bp = new Derived;
    bp->show(); // RUN-TIME
    POLYMORPHISM
    return 0;
}
```

POINTERS

```
int a;  
  
int *pv;      * indirection operator/  
pv=&a;        dereference operator  
a=10;  
cout<<*pv;  
*pv=15;  
cout<<a;
```

The reference operator (&) returns the variable's address.

The dereference operator (*) helps us get the value that has been stored in a memory address.

```
#include<iostream>  
  
using namespace std;  
  
int main()  
{  
    int a=10,*ptr1,**ptr2;  
    ptr1=&a;  
    ptr2=&ptr1;  
    cout<<"A="<<a;  
    cout<<"\nA using ptr1="<<*ptr1;  
    cout<<"\nA using ptr2="<<**ptr2;  
    cout<<"Address of A : "<<ptr1<<endl;  
    cout<<"Address of ptr1 : " <<ptr2<<endl;  
    cout<<"\nManipulation of Pointer:"<<(*ptr1)/2;  
    ptr1+=2;  
    cout<<"Address of A : "<<ptr1<<endl;  
    ptr2+=2;  
    cout<<"Address of ptr1 : " <<ptr2<<endl;  
    cout<<"A="<<a;  
    cout<<"\nA using ptr1="<<*ptr1;  
    cout<<"\nA using ptr2="<<**ptr2;  
}
```

POINTER MANIPULATION

- A pointer can be incremented(++) or decremented(--)
- Any integer can be added to or subtracted from a pointer
- One pointer can be subtracted from another
- An array name contains the address of first element of the array which acts like constant pointer.
- It means, the address stored in array name can't be changed.

For example, if we have an array named **val** then **val** and **&val[0]** can be used interchangeably.

- **int a[6];**
- **int *aptr;**
- **aptr=&a[0];** or **aptr=a;**

//aptr refers to the base address of the variable

POINTERS WITH ARRAYS AND STRINGS

- **Pointer is a tool used to access elements of an array.**
- **Useful to allocate arrays dynamically.**
- **Represents the collection of Addresses.**

Eg. `int *inarray[10];`

```
#include <iostream>
using namespace std;
int main()
{
    char *names[5] = {"john",
                      "Peter",
                      "Marco",
                      "Devin",
                      "Ronan"};
    for(int i=0;i<5;i++)
    {
        cout << names[i] << endl;
    }
    return 0;
}
```

MEMORY MANAGEMENT OPERATOR

- **malloc(), calloc()** - functions to allocate memory dynamically at run time
- **free()** – to free dynamically allocated memory
- In addition two more operators introduced – **new** and **delete**
- **new** - allocates space
- **delete** – deallocates space
- Object can be created by using **new** and destroyed using **delete**.
- Object created inside a block with **new** will remain in existence until it is explicitly destroyed by using **delete**.

MEMORY MANAGEMENT OPERATOR

Syntax : pointer-variable = new datatype;

Eg;

p=new int;

q=new float;

Can have initial values as follows:

ptr = new data_type(value);

value – initial value

Eg:

int *p=new int(25);

float *q = new float(7.5);

MEMORY MANAGEMENT OPERATOR

- new can be used to create memory space for any datatype including array, structure, classes.
- For 1-D array
 - `int *p = new int[10];`
- For 2-D array
 - `int *p = new int[5][4]`
 - here only first size can be variable, all other should be constant sizes.
 - `int *p = new int[n][5];`
- When a pointer is no longer needed may delete to reallocate.
- `delete p;`
- `delete [size] p; //when size specified`

ADVANTAGES OF MMO

Memory management operator new and delete has the following advantages :

- 1. Automatically computes size of data object.(no need of sizeof() operator)**
- 2. Automatically returns correct pointer type.(no need of typecast operator)**
- 3. Possible to initialize object while allocating space.**
- 4. New and delete can be overloaded.**

POINTERS TO FUNCTIONS

- **Used for dynamic binding**
- **Pointer to function is known as callback function**
- Function pointer is a pointer used to point functions.
- It allows to select a function dynamically at run time.
- We can call the function by using the function pointer, or we can also pass the pointer to another function as a parameter.
- They are mainly useful for event-driven applications, callbacks, and even for storing the functions in arrays.
- Syntax:
 - `Data_type (*pointer name)();`

POINTERS TO FUNCTIONS

```
#include <iostream>
using namespace std;
int add(int a , int b)
{
    return a+b;
}
int main()
{
    int (*funcptr)(int,int);
    funcptr=add;
    int sum=funcptr(5,5);
    cout << "value of sum is :" <<sum;
    return 0;
}
```

POINTER TO OBJECTS

Points user-defined objects.

- Eg:
- class apple
- {
- int x;
- };
- apple a;
 - apple *aptr;
 - aptr=&a;
 - aptr->x=10;//access using arrow operator
- (*aptr).x=20;
- apple *bptr = new apple;
- bptr->x=40;
- apple *cptr = new apple[10];
- cptr[0]->x=25;
- apple b, c[10];

CONSTANT POINTER

```
int x=10;
```

```
int *const pv=&x;
```

Here we cannot change the address

```
int y;
```

```
int const *pv=& y;
```

Here we cannot change value of constant

VIRTUAL FUNCTION

- **A virtual function is a member function which is declared within a base class and is re-defined (overridden) by a derived class.**
- **When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.**

VIRTUAL FUNCTION

- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call.
- They are mainly used to achieve Runtime polymorphism
- Functions are declared with a virtual keyword in base class.
- The resolving of function call is done at runtime.

RULES FOR VIRTUAL FUNCTIONS

- Virtual functions cannot be static.
- A virtual function can be a friend function of another class.
- Virtual functions should be accessed using pointer or reference of base class type to achieve runtime polymorphism.
- The prototype of virtual functions should be the same in the base as well as derived class.
- They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or re-define the virtual function), in that case, the base class version of the function is used.
- A class may have virtual destructor but it cannot have a virtual constructor.

EXAMPLE

```
#include<iostream>

using namespace std;

class base {
public:
    virtual void print()
    {
        cout << "print base class\n";
    }

    void show()
    {
        cout << "show base class\n";
    }
};
```

```
class derived : public base {
public:
    void print()
    {
        cout << "print derived class\n";
    }

    void show()
    {
        cout << "show derived class\n";
    }
};

int main()
{
    base *bptr;
    derived d;
    bptr = &d;

    // Virtual function, binded at runtime
    bptr->print();

    // Non-virtual function, binded at compile time
    bptr->show();

    return 0;
}
```

PURE VIRTUAL FUNCTION

A pure virtual function is a function that must be overridden in a derived class and need not be defined in base.

A virtual function is declared to be “pure” using the curious =0 syntax.

```
#include<iostream>

using namespace std;

class Base
{
int x;
public:
    virtual void fun() = 0;
    int getX() { return x; }
};
```

```
// This class inherits from Base and
// implements fun()
class Derived: public Base
{
    int y;
public:
    void fun() { cout << "fun() called";
}
};

int main(void)
{
    Derived d;
    d.fun();
    return 0;
}
```

PURE VIRTUAL FUNCTION

- **A pure virtual function is a function declared in a base class that has no definition relative to the base class.**
- **The compiler expects the derived classes to define that or re-declare it as pure virtual function.**
- **We can't instantiate a class with pure virtual function. So such classes are called as abstract base classes.**
- **Base class is used to provide blue print and that are defined in derived.**
- **Base class is used to create a base pointer for achieving run time polymorphism.**